

Description | Editorial | Solutions | Submissions

3099. Harshad Number

Solved 

Easy Topics Companies Hint

An integer divisible by the **sum** of its digits is said to be a **Harshad** number. You are given an integer `x`. Return the *sum of the digits of `x`* if `x` is a **Harshad** number, otherwise, return `-1`.

Example 1:

Input: `x = 18`

Output: `9`

Explanation:

The sum of digits of `x` is `9`. `18` is divisible by `9`. So `18` is a Harshad number and the answer is `9`.

Example 2:

Input: `x = 23`

Output: `-1`

Explanation:

The sum of digits of `x` is `5`. `23` is not divisible by `5`. So `23` is not a Harshad

227 25 ☆ ↗ ⓘ

</> Code Note X

Java Auto

```
1 class Solution {
2     public int sumOfTheDigitsOfHarshadNumber(int x) {
3         int n=x;
4         int sum=0;
5         while(n>0)
6         {
7             int ans=n%10;
8             sum=sum+ans;
9             n=n/10;
10        }
11        if(x%sum==0)
12            return sum;
13        return -1;
14    }
15 }
```

Saved

☒ Testcase ☒ Test Result

You must run your code first

Problem List

2520. Count the Digits That Divide a Number

Solved

Easy Topics Companies Hint

Given an integer `num`, return the number of digits in `num` that divide `num`.

An integer `val` divides `nums` if `nums % val == 0`.

Example 1:

Input: `num = 7`
Output: `1`
Explanation: 7 divides itself, hence the answer is 1.

Example 2:

Input: `num = 121`
Output: `2`
Explanation: 121 is divisible by 1, but not 2. Since 1 occurs twice as a digit, we return 2.

Example 3:

Input: `num = 1248`
Output: `4`
Explanation: 1248 is divisible by all of its digits,

698 38 4 Online

Code Note X

Java Auto

```
1 class Solution {
2     public int countDigits(int n) {
3         int count=0;
4         int div=1;
5         while(div<=n)
6         {
7             int ans=(n%(div*10))/div;
8             if(n%ans==0)
9                 count++;
10            div=div*10;
11        }
12        return count;
13    }
14 }
```

Saved

Testcase Test Result

You must run your code first

Problem List

<

>

↔

🔍

Submit

📄

🌟

🏠

Description

Editorial

Solutions

Submissions

2652. Sum Multiples

Solved

Easy

Topics

Companies

Hint

Given a positive integer n , find the sum of all integers in the range $[1, n]$ inclusive that are divisible by 3, 5, or 7.

Return an integer denoting the sum of all numbers in the given range satisfying the constraint.

Example 1:

Input: $n = 7$

Output: 21

Explanation: Numbers in the range $[1, 7]$ that are divisible by 3, 5, or 7 are 3, 5, 6, 7. The sum of these numbers is 21.

Example 2:

Input: $n = 10$

Output: 40

Explanation: Numbers in the range $[1, 10]$ that are divisible by 3, 5, or 7 are 3, 5, 6, 7, 9, 10. The sum of these numbers is 40.

Example 3:

601

🗣️

37

☆

📄

?

2 Online

</> Code

Note

Java

Auto

```
1 class Solution {
2     public int sumOfMultiples(int n) {
3         int sum=0;
4         for(int i=1;i<=n;i++)
5         {
6             if(i%3==0||i%5==0||i%7==0)
7             {
8                 sum=sum+i;
9             }
10        }
11        return sum;
12    }
13 }
```

Saved

☒ Testcase

☒ Test Result

You must run your code first

2485. Find the Pivot Integer

Solved ✓

Easy Topics Companies Hint

Given a positive integer n , find the **pivot integer** x such that:

- The sum of all elements between 1 and x inclusively equals the sum of all elements between x and n inclusively.

Return the *pivot integer* x . If no such integer exists, return -1 . It is guaranteed that there will be at most one pivot index for the given input.

Example 1:

Input: $n = 8$

Output: 6

Explanation: 6 is the pivot integer since: $1 + 2 + 3 + 4 + 5 + 6 = 6 + 7 + 8 = 21$.

Example 2:

Input: $n = 1$

Output: 1

Explanation: 1 is the pivot integer since: $1 = 1$.

Example 3:

Input: $n = 4$

1.5K 116 ☆ ↗ ⓘ

</> Code Note X

Java Auto

```
1 class Solution {
2     public int pivotInteger(int n) {
3         int total=n*(n+1)/2;
4         int leftsum=0;
5         for(int i=1;i<=n;i++)
6         {
7             leftsum+=i;
8             if(leftsum==total-leftsum+i)
9             {
10                 return i;
11             }
12         }
13         return -1;
14     }
15 }
```

Saved

✓ Testcase > Test Result

You must run your code first

2160. Minimum Sum of Four Digit Number After Splitting Digits

Solved

Easy Topics Companies Hint

You are given a **positive** integer `num` consisting of exactly four digits. Split `num` into two new integers `new1` and `new2` by using the **digits** found in `num`.

Leading zeros are allowed in `new1` and `new2`, and **all** the digits found in `num` must be used.

- For example, given `num = 2932`, you have the following digits: two `2`'s, one `9` and one `3`. Some of the possible pairs `[new1, new2]` are `[22, 93]`, `[23, 92]`, `[223, 9]` and `[2, 329]`.

Return the **minimum** possible sum of `new1` and `new2`.

Example 1:

Input: `num = 2932`

Output: 52

Explanation: Some possible pairs `[new1, new2]` are `[29, 23]`, `[223, 9]`, etc.

The minimum sum can be obtained by the pair `[29, 23]`: `29 + 23 = 52`.

Example 2:

1.5K 49 5 Online

Code Note

Java Auto

```
1 class Solution {
2     public int minimumSum(int num) {
3         int[] digits = new int[4];
4         for (int i = 3; i >= 0; i--)
5         {
6             digits[i] = num % 10;
7             num = num / 10;
8         }
9         Arrays.sort(digits);
10        int num1 = digits[0] * 10 + digits[2];
11        int num2 = digits[1] * 10 + digits[3];
12        return num1 + num2;
13    }
14 }
```

Saved

Testcase Test Result

2160. Minimum Sum of Four Digit Number After Splitting Digits

Solved

Easy Topics Companies Hint

You are given a **positive** integer `num` consisting of exactly four digits. Split `num` into two new integers `new1` and `new2` by using the **digits** found in `num`. **Leading zeros** are allowed in `new1` and `new2`, and **all** the digits found in `num` must be used.

- For example, given `num = 2932`, you have the following digits: two `2`'s, one `9` and one `3`. Some of the possible pairs `[new1, new2]` are `[22, 93]`, `[23, 92]`, `[223, 9]` and `[2, 329]`.

Return the **minimum** possible sum of `new1` and `new2`.

Example 1:

Input: `num = 2932`

Output: `52`

Explanation: Some possible pairs `[new1, new2]` are `[29, 23]`, `[223, 9]`, etc.

The minimum sum can be obtained by the pair `[29, 23]`: `29 + 23 = 52`.

Example 2:

1.5K 49 5 Online

Java Auto

```
1 class Solution {
2     public int minimumSum(int num) {
3         int[] digits = new int[4];
4         for (int i = 3; i >= 0; i--)
5         {
6             digits[i] = num % 10;
7             num = num / 10;
8         }
9         Arrays.sort(digits);
10        int num1 = digits[0] * 10 + digits[2];
11        int num2 = digits[1] * 10 + digits[3];
12        return num1 + num2;
13    }
14 }
```

Saved

Testcase Test Result

You must run your code

 Problem List < > 🔍

Description

Editorial

Solutions

Submissions

2235. Add Two Integers

Solved

Easy

Topics

Companies

Given two integers `num1` and `num2`, return the **sum** of the two integers.

Example 1:

Input: `num1 = 12, num2 = 5`

Output: `17`

Explanation: `num1` is 12, `num2` is 5, and their sum is 12 + 5 = 17, so 17 is returned.

Example 2:

Input: `num1 = -10, num2 = 4`

Output: `-6`

Explanation: `num1 + num2 = -6`, so -6 is returned.

Constraints:

- `-100 ≤ num1, num2 ≤ 100`

2.2K

435

☆

🔗

?

</> Code

Note

×

Java 🔻 🔒 Auto

```
1 class Solution {
2     public int sum(int num1, int num2) {
3         return num1+ num2;
4     }
5 }
```

Saved

☑ Testcase

> Test Result

You must run your code first

 Problem List < > 🔍

[Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)  <

2119. A Number After a Double Reversal Solved

Easy Topics Companies Hint

Reversing an integer means to reverse all its digits.

- For example, reversing 2021 gives 1202. Reversing 12300 gives 321 as the **leading zeros are not retained**.

Given an integer `num`, **reverse** `num` to get `reversed1`, **then reverse** `reversed1` to get `reversed2`. Return `true` if `reversed2 equals num`. Otherwise return `false`.

Example 1:

Input: `num = 526`

Output: `true`

Explanation: Reverse `num` to get 625, then reverse 625 to get 526, which equals `num`.

Example 2:

Input: `num = 1800`

Output: `false`

Explanation: Reverse `num` to get 81, then reverse 81 to get 18, which does not equal `num`.

808  56    8 Online

  Submit  

</> Code | Note X

Java ▾ Auto

```
1 class Solution {
2     public boolean isSameAfterReversals(int n) {
3         int count=0;
4         int div=1;
5         if(n%10!=0||n==0)
6         {
7             return true;
8         }
9         return false;
10    }
11 }
```

Saved

☒ Testcase | >_ Test Result

You must run your code first

Problem List

<

>

🔍

🔖 Description

📖 Editorial

🔗 Solutions

🔄 Submissions

1929. Concatenation of Array

Solved

Easy

Topics

Companies

Hint

Given an integer array `nums` of length `n`, you want to create an array `ans` of length `2n` where `ans[i] == nums[i]` and `ans[i + n] == nums[i]` for $0 \leq i < n$ (**0-indexed**).

Specifically, `ans` is the **concatenation** of two `nums` arrays.

Return *the array* `ans`.

Example 1:

Input: `nums = [1,2,1]`

Output: `[1,2,1,1,2,1]`

Explanation: The array `ans` is formed as follows:

– `ans =`

`[nums[0], nums[1], nums[2], nums[0], nums[1], nums[2]]`

– `ans = [1,2,1,1,2,1]`

Example 2:

Input: `nums = [1,3,2,1]`

Output: `[1,3,2,1,1,3,2,1]`

Explanation: The array `ans` is formed as follows:

– `ans =`

`[nums[0], nums[1], nums[2], nums[3], nums[0], nums[1], nums[2], nums[3]]`

4.3K

🔊

236

☆

🔗

?

64 Online

</> Code

Note X

Java

🔒 Auto

```
1 class Solution {
2     public int[] getConcatenation(int[] nums) {
3         int n=nums.length;
4         int ans[]=new int[n*2];
5         for(int i=0;i<n;i++)
6         {
7             ans[i]=nums[i];
8             ans[i+n]=nums[i];
9         }
10        return ans;
11    }
12 }
```

Saved

☑ Testcase

> Test Result

You must run your code first

Problem List < > 🔍

Description | Editorial | Solutions | Submissions

1920. Build Array from Permutation Solved

Easy Topics Companies Hint

Given a **zero-based permutation** `nums` (**0-indexed**), build an array `ans` of the **same length** where `ans[i] = nums[nums[i]]` for each `0 ≤ i < nums.length` and return it.

A **zero-based permutation** `nums` is an array of **distinct** integers from `0` to `nums.length - 1` (**inclusive**).

Example 1:

Input: `nums = [0,2,1,5,3,4]`
Output: `[0,1,2,4,5,3]`
Explanation: The array `ans` is built as follows:
`ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]],`
`nums[nums[3]], nums[nums[4]], nums[nums[5]]`
`= [nums[0], nums[2], nums[1], nums[5], nums[3],`
`nums[4]]`
`= [0,1,2,4,5,3]`

Example 2:

Input: `nums = [5,0,1,2,3,4]`
Output: `[4,5,0,1,2,3]`
Explanation: The array `ans` is built as follows:
`ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]],`
`nums[nums[3]], nums[nums[4]], nums[nums[5]]`
`= [nums[5], nums[0], nums[1], nums[2], nums[3],`
`nums[4]]`
`= [4,5,0,1,2,3]`

4.1K 422 18 Online

Submit

</> Code | Note X

Java ▾ Auto

```
1 class Solution {
2     public int[] buildArray(int[] nums) {
3         int ans[] = new int[nums.length];
4         for (int i = 0; i < nums.length; i++)
5         {
6             ans[i] = nums[nums[i]];
7         }
8         return ans;
9     }
10 }
```

Saved

☒ Testcase Test Result

You must run your code first

Problem List

1450. Number of Students Doing Homework at a Given Time

Solved

EasyTopicsCompaniesHint

Given two integer arrays `startTime` and `endTime` and given an integer `queryTime`.
The i th student started doing their homework at the time `startTime[i]` and finished it at time `endTime[i]`.
Return the number of students doing their homework at time `queryTime`. More formally, return the number of students where `queryTime` lays in the interval `[startTime[i], endTime[i]]` inclusive.

Example 1:
Input: `startTime = [1,2,3]`, `endTime = [3,2,7]`, `queryTime = 4`
Output: 1
Explanation: We have 3 students where:
The first student started doing homework at time 1 and finished at time 3 and wasn't doing anything at time 4.
The second student started doing homework at time 2 and finished at time 2 and also wasn't doing anything at time 4.
The third student started doing homework at time 3 and

940201Online

CodeNote

JavaAuto

```
1 class Solution {
2     public int busyStudent(int[] startTime, int[] endTime, int queryTime) {
3         int count=0;
4         for(int i=0;i<startTime.length;i++)
5         {
6             if(startTime[i]<=queryTime && queryTime<=endTime[i])
7             {
8                 count++;
9             }
10        }
11        return count;
12    }
13 }
```

Saved

TestcaseTest Result

You must run your code first

Problem List

<

>

🔍

Description

Editorial

Solutions

Submissions

1470. Shuffle the Array

Solved

Easy

Topics

Companies

Hint

Given the array `nums` consisting of `2n` elements in the form $[x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n]$.

Return the array in the form $[x_1, y_1, x_2, y_2, \dots, x_n, y_n]$.

Example 1:

Input: `nums = [2,5,1,3,4,7]`, `n = 3`

Output: `[2,3,5,4,1,7]`

Explanation: Since $x_1=2$, $x_2=5$, $x_3=1$, $y_1=3$, $y_2=4$, $y_3=7$ then the answer is `[2,3,5,4,1,7]`.

Example 2:

Input: `nums = [1,2,3,4,4,3,2,1]`, `n = 4`

Output: `[1,4,2,3,3,2,4,1]`

Example 3:

Input: `nums = [1,1,2,2]`, `n = 2`

Output: `[1,2,1,2]`

6.5K

164

☆

🔗

🔄

🚀

▶

Submit

📄

🌟

</> Code

Note

×

Java

Auto

```
1 class Solution {
2     public int[] shuffle(int[] nums, int n) {
3         int[] ans = new int[n * 2];
4         for (int i = 0; i < n; i++) {
5             ans[i * 2] = nums[i];
6             ans[i * 2 + 1] = nums[n + i];
7         }
8         return ans;
9     }
10 }
```

Saved

☑ Testcase

>_ Test Result

You must run your code first

Problem List < > 🔍

Description

Editorial

Solutions

Submissions

292. Nim Game

Solved

Easy Topics Companies Hint

You are playing the following Nim Game with your friend:

- Initially, there is a heap of stones on the table.
- You and your friend will alternate taking turns, and **you go first**.
- On each turn, the person whose turn it is will remove 1 to 3 stones from the heap.
- The one who removes the last stone is the winner.

Given `n`, the number of stones in the heap, return `true` if you can win the game assuming both you and your friend play optimally, otherwise return `false`.

Example 1:

Input: `n = 4`

Output: `false`

Explanation: These are the possible outcomes:

- You remove 1 stone. Your friend removes 3 stones, including the last stone. Your friend wins.
- You remove 2 stones. Your friend removes 2 stones, including the last stone. Your friend wins.
- You remove 3 stones. Your friend removes the last

2K 116 7 Online

Submit

</> Code

Note X

Java Auto

```
1 class Solution {
2     public boolean canWinNim(int n) {
3         if(n%4==0)
4         {
5             return false;
6         }
7         return true;
8     }
9 }
```

Saved

Testcase

Test Result

You must run your code first

Description Editorial Solutions Submissions

1323. Maximum 69 Number

Solved ✓

Easy Topics Companies Hint

You are given a positive integer `num` consisting only of digits `6` and `9`.

Return the maximum number you can get by changing **at most** one digit (`6` becomes `9`, and `9` becomes `6`).

Example 1:

Input: `num = 9669`

Output: `9969`

Explanation:

Changing the first digit results in `6669`.
 Changing the second digit results in `9969`.
 Changing the third digit results in `9699`.
 Changing the fourth digit results in `9666`.
 The maximum number is `9969`.

Example 2:

Input: `num = 9996`

Output: `9999`

Explanation: Changing the last digit `6` to `9` results in the maximum number.

Example 3:

👍 3.3K 🗨️ 428 ☆ 📌 ?

5 Online

</> Code Note X

Java Auto

```
1 class Solution {
2     public int maximum69Number (int num) {
3         String s=Integer.toString(num);
4         char[]arr=s.toCharArray();
5         for(int i=0;i<arr.length;i++)
6         {
7             if(arr[i]=='6')
8             {
9                 arr[i]='9';
10                break;
11            }
12        }
13        return Integer.parseInt(new String(arr));
14    }
15 }
```

Saved

☑️ Testcase ➤ Test Result

You must run your code first

Problem List

1234567891011121314151617181920212223242526272829303132333435363738394041424344454647484950515253545556575859606162636465666768697071727374757677787980818283848586878889909192939495969798991001011021031041051061071081091101111121131141151161171181191201211221231241251261271281291301311321331341351361371381391401411421431441451461471481491501511521531541551561571581591601611621631641651661671681691701711721731741751761771781791801811821831841851861871881891901911921931941951961971981992002012022032042052062072082092102112122132142152162172182192202212222232242252262272282292302312322332342352362372382392402412422432442452462472482492502512522532542552562572582592602612622632642652662672682692702712722732742752762772782792802812822832842852862872882892902912922932942952962972982993003013023033043053063073083093103113123133143153163173183193203213223233243253263273283293303313323333343353363373383393403413423433443453463473483493503513523533543553563573583593603613623633643653663673683693703713723733743753763773783793803813823833843853863873883893903913923933943953963973983994004014024034044054064074084094104114124134144154164174184194204214224234244254264274284294304314324334344354364374384394404414424434444454464474484494504514524534544554564574584594604614624634644654664674684694704714724734744754764774784794804814824834844854864874884894904914924934944954964974984995005015025035045055065075085095105115125135145155165175185195205215225235245255265275285295305315325335345355365375385395405415425435445455465475485495505515525535545555565575585595605615625635645655665675685695705715725735745755765775785795805815825835845855865875885895905915925935945955965975985996006016026036046056066076086096106116126136146156166176186196206216226236246256266276286296306316326336346356366376386396406416426436446456466476486496506516526536546556566576586596606616626636646656666676686696706716726736746756766776786796806816826836846856866876886896906916926936946956966976986997007017027037047057067077087097107117127137147157167177187197207217227237247257267277287297307317327337347357367377387397407417427437447457467477487497507517527537547557567577587597607617627637647657667677687697707717727737747757767777787797807817827837847857867877887897907917927937947957967977987998008018028038048058068078088098108118128138148158168178188198208218228238248258268278288298308318328338348358368378388398408418428438448458468478488498508518528538548558568578588598608618628638648658668678688698708718728738748758768778788798808818828838848858868878888898908918928938948958968978988999009019029039049059069079089099109119129139149159169179189199209219229239249259269279289299309319329339349359369379389399409419429439449459469479489499509519529539549559569579589599609619629639649659669679689699709719729739749759769779789799809819829839849859869879889899909919929939949959969979989991000100110021003100410051006100710081009101010111012101310141015101610171018101910201021102210231024102510261027102810291030103110321033103410351036103710381039104010411042104310441045104610471048104910501051105210531054105510561057105810591060106110621063106410651066106710681069107010711072107310741075107610771078107910801081108210831084108510861087108810891090109110921093109410951096109710981099110011011102110311041105110611071108110911101111111211131114111511161117111811191120112111221123112411251126112711281129113011311132113311341135113611371138113911401141114211431144114511461147114811491150115111521153115411551156115711581159116011611162116311641165116611671168116911701171117211731174117511761177117811791180118111821183118411851186118711881189119011911192119311941195119611971198119912001201120212031204120512061207120812091210121112121213121412151216121712181219122012211222122312241225122612271228122912301231123212331234123512361237123812391240124112421243124412451246124712481249125012511252125312541255125612571258125912601261126212631264126512661267126812691270127112721273127412751276127712781279128012811282128312841285128612871288128912901291129212931294129512961297129812991300130113021303130413051306130713081309131013111312131313141315131613171318131913201321132213231324132513261327132813291330133113321333133413351336133713381339134013411342134313441345134613471348134913501351135213531354135513561357135813591360136113621363136413651366136713681369137013711372137313741375137613771378137913801381138213831384138513861387138813891390139113921393139413951396139713981399140014011402140314041405140614071408140914101411141214131414141514161417141814191420142114221423142414251426142714281429143014311432143314341435143614371438143914401441144214431444144514461447144814491450145114521453145414551456145714581459146014611462146314641465146614671468146914701471147214731474147514761477147814791480148114821483148414851486148714881489149014911492149314941495149614971498149915001501150215031504150515061507150815091510151115121513151415151516151715181519152015211522152315241525152615271528152915301531153215331534153515361537153815391540154115421543154415451546154715481549155015511552155315541555155615571558155915601561156215631564156515661567156815691570157115721573157415751576157715781579158015811582158315841585158615871588158915901591159215931594159515961597159815991600160116021603160416051606160716081609161016111612161316141615161616171618161916201621162216231624162516261627162816291630163116321633163416351636163716381639164016411642164316441645164616471648164916501651165216531654165516561657165816591660166116621663166416651666166716681669167016711672167316741675167616771678167916801681168216831684168516861687168816891690169116921693169416951696169716981699170017011702170317041705170617071708170917101711171217131714171517161717171817191720172117221723172417251726172717281729173017311732173317341735173617371738173917401741174217431744174517461747174817491750175117521753175417551756175717581759176017611762176317641765176617671768176917701771177217731774177517761777177817791780178117821783178417851786178717881789179017911792179317941795179617971798179918001801180218031804180518061807180818091810181118121813181418151816181718181819182018211822182318241825182618271828182918301831183218331834183518361837183818391840184118421843184418451846184718481849185018511852185318541855185618571858185918601861186218631864186518661867186818691870187118721873187418751876187718781879188018811882188318841885188618871888188918901891189218931894189518961897189818991900190119021903190419051906190719081909191019111912191319141915191619171918191919201921192219231924192519261927192819291930193119321933193419351936193719381939194019411942194319441945194619471948194919501951195219531954195519561957195819591960196119621963196419651966196719681969197019711972197319741975197619771978197919801981198219831984198519861987198819891990199119921993199419951996199719981999200020012002200320042005200620072008200920102011201220132014201520162017201820192020202120222023202420252026202720282029203020312032203320342035203620372038203920402041204220432044204520462047204820492050205120522053205420552056205720582059206020612062206320642065206620672068206920702071207220732074207520762077207820792080208120822083208420852086208720882089209020912092209320942095209620972098209921002101210221032104210521062107210821092110211121122113211421152116211721182119212021212122212321242125212621272128212921302131213221332134213521362137213821392140214121422143214421452146214721482149215021512152215321542155215621572158215921602161216221632164216521662167216821692170217121722173217421752176217721782179218021812182218321842185218621872188218921902191219221932194219521962197219821992200220122022203220422052206220722082209221022112212221322142215221622172218221922202221222222232224222522262227222822292230223122322233223422352236223722382239224022412242224322442245224622472248224922502251225222532254225522562257225822592260226122622263226422652266226722682269227022712272227322742275227622772278227922802281228222832284228522862287228822892290229122922293229422952296229722982299230023012302230323042305230623072308230923102311231223132314231523162317231823192320232123222323232423252326232723282329233023312332233323342335233623372338233923402341234223432344234523462347234823492350235123522353235423552356235723582359236023612362236323642365236623672368236923702371237223732374237523762377237823792380238123822383238423852386238723882389239023912392239323942395239623972398239924002401240224032404240524062407240824092410241124122413241424152416241724182419242024212422242324242425242624272428242924302431243224332434243524362437243824392440244124422443244424452446244724482449245024512452245324542455245624572458245924602461246224632464246524662467246824692470247124722473247424752476247724782479248024812482248324842485248624872488248924902491249224932494249524962497249824992500250125022503250425052506250725082509251025112512251325142515251625172518251925202521252225232524252525262527252825292530253125322533253425352536253725382539254025412542254325442545254625472548254925502551255225532554255525562557255825592560256125622563256425652566256725682569257025712572257325742575257625772578257925802581258225832584258525862587258825892590259125922593259425952596259725982599260026012602260326042605260626072608260926102611261226132614261526162617261826192620262126222623262426252626262726282629263026312632263326342635263626372638263926402641264226432644264526462647264826492650265126522653265426552656265726582659266026612662266326642665266626672668266926702671267226732674267526762677267826792680268126822683268426852686268726882689269026912692269326942695269626972698269927002701270227032704270527062707270827092710271127122713271427152716271727182719272027212722272327242725272627272728272927302731273227332734273527362737273827392740274127422743274427452746274727482749275027512752275327542755275627572758275927602761276227632764276527662767276827692770277127722773277427752776277727782779278027812782278327842785278627872788278927902791279227932794279527962797279827992800280128022803280428052806280728082809281028112812281328142815281628172818281928202821282228232824282528262827282828292830283128322833283428352836283728382839284028412842284328442845284628472848284928502851285228532854285528562857285828592860286128622863286428652866286728682869287028712872287328742875287628772878287928802881288228832884288528862887288828892890289128922893289428952896289728982899290029012902290329042905290629072908290929102911291229132914291529162917291829192920292129222923292429252926292729282929293029312932293329342935293629372938293929402941294229432944294529462947294829492950295129522953295429552956295729582959296029612962296329642965296629672968296929702971297229732974297529762977297829792980298129822983298429852986298729882989299029912992299329942995299629972998299930003001300230033004300530063007300830093010301130123013301430153016301730183019302030213022302330243025302630273028302930303031303230333034303530363037303830393040304130423043304430453046304730483049305030513052305330543055305630573058305930603061306230633064306530663067306830693070307130723073307430753076307730783079308030813082308330843085308630873088308930903091309230933094309530963097309830993100310131023103310431053106310731083109311031113112311331143115311631173118311931203121312231233124312531263127312831293130313131323133313431353136313731383139314031413142314331443145314631473148314931503151315231533154315531563157315831593160316131623163316431653166316731683169317031713172317331743175317631773178317931803181318231833184318531863187318831893190319131923193319431953196319731983199320032013202320332043205320632073208320932103211321232133214321532163217321832193220322132223223322343225322632273228322932303231323232333234323532363237323832393240324132423243324432453246324732483249325032513252325332543255325632573258325932603261326232633264326532663267326832693270327132723273327432753276327732783279328032813282328332843285328632873288328932903291329232933294329532963297329832993300330133023303330433053306330733083309331033113312331333143315331633173318331933203321332233233324332533263327332833293330333133323333333433353336333733383339334033413342334333443345334633473348334933503351335233533354335533563357335833593360336133623363

Problem List

1281. Subtract the Product and Sum of Digits of an Integer

Solved

Easy

Topics

Companies

Hint

Given an integer number n , return the difference between the product of its digits and the sum of its digits.

Example 1:

Input: $n = 234$
Output: 15
Explanation:
Product of digits = $2 * 3 * 4 = 24$
Sum of digits = $2 + 3 + 4 = 9$
Result = $24 - 9 = 15$

Example 2:

Input: $n = 4421$
Output: 21
Explanation:
Product of digits = $4 * 4 * 2 * 1 = 32$
Sum of digits = $4 + 4 + 2 + 1 = 11$
Result = $32 - 11 = 21$

2.9K

94

Code

Note

Java

Auto

```
1 class Solution {
2     public int subtractProductAndSum(int n) {
3         int sum=0;
4         int prod=1;
5         while(n>0)
6         {
7             int ld=n%10;
8             sum=sum+ld;
9             prod=prod*ld;
10            n=n/10;
11        }
12        return prod-sum;
13    }
14 }
```

Saved

Testcase

Test Result

You must run your code first

27. Remove Element

Solved 

Easy Topics Companies Hint

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` **in-place**. The order of the elements may be changed. Then return *the number of elements in `nums` which are not equal to `val`*.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with
                             // correct length.
                             // It is sorted with no
                             // values equaling val.

int k = removeElement(nums, val); // Calls your
```

5.2K 965

Upgrade to premium to use autocomplete

Java Auto

```

1 class Solution {
2     public int removeElement(int[] nums, int val) {
3         int k=0;
4         for(int i=0;i<nums.length;i++)
5         {
6             if(nums[i]!=val)
7             {
8                 nums[k]=nums[i];
9                 k++;
10            }
11        }
12        return k;
13    }
14 }

```

Saved

Ln 1, Col 1

Testcase > Test Result

Problem List

DescriptionEditorialSolutionsSubmissions

66. Plus One

Solved

EasyTopicsCompanies

You are given a **large integer** represented as an integer array `digits`, where each `digits[i]` is the i^{th} digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's.

Increment the large integer by one and return *the resulting array of digits*.

Example 1:

Input: `digits = [1,2,3]`

Output: `[1,2,4]`

Explanation: The array represents the integer 123. Incrementing by one gives $123 + 1 = 124$. Thus, the result should be `[1,2,4]`.

Example 2:

Input: `digits = [4,3,2,1]`

Output: `[4,3,2,2]`

Explanation: The array represents the integer 4321. Incrementing by one gives $4321 + 1 = 4322$. Thus, the result should be `[4,3,2,2]`.

Example 3:

Input: `digits = [9]`

Output: `[1,0]`

Explanation: The array represents the integer 9. Incrementing by one gives $9 + 1 = 10$. Thus, the result should be `[1,0]`.

11.9K703113 Online

CodeNote

JavaAuto

```
1 class Solution {
2     public int[] plusOne(int[] digits) {
3         for(int i=digits.length-1;i>=0;i--)
4         {
5             if(digits[i]!=9)
6             {
7                 digits[i]=digits[i]+1;
8                 return digits;
9             }
10            digits[i]=0;
11        }
12        int ans[]=new int[digits.length+1];
13        ans[0]=1;
14        return ans;
15    }
16 }
17
```

SavedLn 1, Col 1

TestcaseTest Result

You must run your code first

Problem List

292. Nim Game

Solved

Easy

Topics

Companies

Hint

You are playing the following Nim Game with your friend:

- Initially, there is a heap of stones on the table.
- You and your friend will alternate taking turns, and **you go first**.
- On each turn, the person whose turn it is will remove 1 to 3 stones from the heap.
- The one who removes the last stone is the winner.

Given `n`, the number of stones in the heap, return `true` if you can win the game assuming both you and your friend play optimally, otherwise return `false`.

Example 1:

Input: `n = 4`
Output: `false`
Explanation: These are the possible outcomes:
1. You remove 1 stone. Your friend removes 3 stones, including the last stone. Your friend wins.
2. You remove 2 stones. Your friend removes 2 stones, including the last stone. Your friend wins.
3. You remove 3 stones. Your friend removes the last

7 Online

Code

Note

Java Auto

```
1 class Solution {
2     public boolean canWinNim(int n) {
3         if(n%4==0)
4         {
5             return false;
6         }
7         return true;
8     }
9 }
```

Saved

Testcase

Test Result

You must run your code first

Problem List

2. Add Two Numbers

Solved

Medium

Topics

Companies

You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:

2

4

3

5

6

4

37K

1.1K

Code

Java

```
1 /**
2  * Definition for singly-linked list.
3  * public class ListNode {
4  *     int val;
5  *     ListNode next;
6  *     ListNode() {}
7  *     ListNode(int val) { this.val = val; }
8  *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
9  * }
10 */
11 class Solution {
12     public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
13         ListNode node=new ListNode(0);
14         ListNode current= node;
15         int carry=0;
16         while(l1!=null||l2!=null||carry!=0)
17         {
18             int sum=carry;
19             if(l1!=null)
20             {
21                 sum+=l1.val;
```

Saved

Ln 1, Col 1

Testcase

Test Result

You must run your code first

Problem List

Description

Editorial

Solutions

Submissions

1. Two Sum

Solved

EasyTopicsCompaniesHint

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to `target`*.

You may assume that each input would have **exactly one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

Input:

`nums = [2,7,11,15], target = 9`

Output:

`[0,1]`

Explanation:

 Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input:

`nums = [3,2,4], target = 6`

Output:

`[1,2]`

Example 3:

Input:

`nums = [3,3], target = 6`

Output:

`[0,1]`

69.2K2K1744 Online

CodeNote

JavaAuto

```
1 class Solution {
2     public int[] twoSum(int[] nums, int target) {
3         HashMap<Integer,Integer>Map=new HashMap();
4         for(int i=0;i<nums.length;i++)
5         {
6             int num=target-nums[i];
7             if(Map.containsKey(num))
8             {
9                 return new int[] {Map.get(num),i};
10            }
11            Map.put(nums[i],i);
12        }
13        return new int[]{};
14    }
15 }
16 }
```

Restored from local Upgrade to Cloud SavingLn 1, Col 1

TestcaseTest Result

You must run your code first

Problem List

DescriptionEditorialSolutionsSubmissions

9. Palindrome Number

Solved

EasyTopicsCompaniesHint

Given an integer x , return `true` if x is a **palindrome**, and `false` otherwise.

Example 1:

Input: $x = 121$
Output: `true`
Explanation: 121 reads as 121 from left to right and from right to left.

Example 2:

Input: $x = -121$
Output: `false`
Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.

Example 3:

Input: $x = 10$
Output: `false`
Explanation: Reads 01 from right to left. Therefore it is not a palindrome.

16.1K742319 Online

CodeNote

JavaAuto

```
1 class Solution {
2     public boolean isPalindrome(int n) {
3         if(n<0)
4         {
5             return false;
6         }
7         int div=n;
8         int rev=0;
9
10        while(n>0)
11        {
12            int ld=n%10;
13            rev=rev*10+ld;
14            n=n/10;
15        }
16        return div==rev;
17    }
18 }
```

SavedLn 1, C

TestcaseTest Result

You must run your code first